

数字IC设计仿真环境使用教程

目录

修订记录

日期	修订版本	描述	作者
2017-02-05	1.0	初稿完成	伍元聪
2018-09-23	1.1	增加pyauto功能脚本	伍元聪
2020-04-16	1.2	增加数字后端综合与形式验证功能脚本	伍元聪
2021-07-01	1.3	增加create_eda_env命令，修改命令描述	伍元聪

数字IC设计仿真环境使用教程

目录

修订记录

建立文件体系

EDA工具Docker环境

数字前端

文件准备

VCS仿真

Verdi查看波形

nLint代码风格检查

仿真环境清理

数字验证

UVM验证

VCS代码覆盖率检查

数字后端

合并RTL设计文件

更新DC综合文件

DC逻辑综合

DC综合结果备份

DC综合环境清理

Formality形式验证

Formality验证环境清理

自动化脚本命令使用说明

Python Env

Makefile Env

清空工程

建立文件体系

create_eda_env

该命令将会自动创建设计环境所需的文件夹和部分所需的文件，包括：

- docs //文档以及相关初始化脚本
- backend //数字后端工作路径

- rtl //RTL设计文件路径
- simfile //前后仿文件列表路径
- testbench //tb文件路径
- nLint //nLint规则文件路径
- tsk //仿真时间尺度文件路径
- fsdb //仿真波形文件路径
- log //日志文件路径

EDA工具Docker环境

```
dsh; eda
```

该命令将进入所有数字IC EDA软件环境，**本文档中所有命令需在EDA Docker环境中执行**

数字前端

文件准备

若要完成整个数字前端仿真流程工作，需在建立完目录后做以下工作

- 将源文件（.v文件）放置到 `./rtl/` 目录下，tb文件（.v/.sv）放置到 `./testbench/` 目录下
- 建议使用自动化生成testbench文件命令：`pyauto tbgen ./rtl/xx.v ./testbench/`
- 根据仿真情形（RTL/PRE/POST）在 `./simfile/` 目录下对应的仿真列表文件中添加参与仿真的文件名（建议使用自动化命令生成 `pyauto simfilegen ./rtl/ ./testbench/xx.v ./simfile/simfile_xx.f`），生成的文件内容格式如下所示：

```
./testbench/tb_file1.sv      #testbench文件
./rtl/file1.v               #rtl顶层文件
./rtl/file2.v               #rtl其他文件
```

- 若要修改仿真时间尺度，在 `./tsk/` 目录下的 `timescale.v` 文件中修改仿真时间尺度规定，默认如

```
`timescale 1ns/1ps
```

VCS仿真

```
make vcs SIMMODE=RTL FSDB_NUM=num
```

该命令将启用VCS仿真，`SIMMODE` 参数为仿真模式选择（RTL级仿真，PRE网表级前仿，POST网表级后仿）缺省默认为 `RTL`，`FSDB_NUM` 参数为仿真波形文件名缺省为空。参加仿真的文件为

```
simfile/simfile_rtl.f
```

文件列表中的文件，产生的报告将保存在 `./log/sim/vcs_rtlsim.log`

Verdi查看波形

```
make verdi
```

该命令启用Verdi查看波形，默认打开fsdb文件夹下的 `rtl_$(FSDB_NUM).fsdb` 文件

```
make verdi FSDB_NUM = num
```

该命令可使用Verdi查看指定的fsdb文件的波形

nLint代码风格检查

```
make nlint
```

该命令将调用nLint对 `./rtl/frontend_file.list` 文件列表中的文件进行代码风格检查，产生的报告保存在 `./log/nlintg/nLint.log` 中

仿真环境清理

```
make clean_sim
```

该命令将执行以下操作：

- 清空log目录下的 `./log/sim/*.log` 文件
- 清空fsdb目录下的 `*.fsdb` 文件
- 清空 `./simv*`
- 清空 `./*.csrc`
- 清空 `./*.rc`
- 清空 `./*.conf`
- 清空 `./*.key`

数字验证

UVM验证

```
make uvm
```

该命令将启用uvm验证，timescale被设为1ns/1ps，原 `timescale.v` 中的timescale失效，在启用前确保 `simfile/simfile_rtl.f` 中包含所有需仿真文件

VCS代码覆盖率检查

```
make cov
```

调用VCS仿真后，使用该命令可产生代码覆盖率检查文档

数字后端

合并RTL设计文件

```
make dc_link TOP=topname
```

该命令将 `./rtl/frontend_file.list`（可使用 `pyauto flstgen ./rtl/./rtl/frontend_file.list` 生成filelist）中包含的文件连接成一个 `.v` 文件，将其保存为 `rel/$(TOP).v`；并运行nLint检查该文件的代码风格，产生的报告保存在 `./log/nlintg/rel_nLint.log` 中

更新DC综合文件

```
make update TOP=topname
```

该命令将 `./backend/re1/$(TOP).v` `./backend/sdc/topfile.func.sdc` `./backend/sdc/io.sdc` 待综合以及约束文件更新至 `./backend/syn/dct/inputs/$(TOP).v`
`./backend/syn/dct/inputs/$(TOP).func.sdc` `./backend/syn/dct/inputs/io.sdc` 对应的文件中

DC逻辑综合

```
make dc TOP=topname
```

该命令将运行DC综合 `./backend/syn/dct/inputs/` 路径下的 `$(TOP).v` 文件，产生的报告保存 `./backend/syn/dct/dc.log` 文件中，在综合前，保证：

- 时序约束文件 `./backend/sdc/topfile.func.sdc` 和IO时序约束文件 `./backend/sdc/io.sdc` 已经根据要求修改
- `./backend/syn/dct/script/` 文件夹下对应的tcl脚本已经根据要求修改
- 确定 `./backend/syn/dct/inputs/` 下的待综合文件已经更新（已运行 `make update TOP=topname`）

DC综合结果备份

```
make backup NAME_BP=backupname
```

该命令将 `./backend/syn/dct/` 综合文件以及综合结果备份至 `./backend/syn/backup/$(NAME_BP)/` 目录下

DC综合环境清理

```
make clean_dct
```

该命令将删除 `./backend/syn/dct/` 目录下产生的临时文件、综合输出结果、报告日志等文件

Formality形式验证

```
make fm_dct TOP=topname
```

该命令将运行Formality对 `./backend/syn/dct/inputs/` 目录下的RTL文件 `$(TOP).v` 和 `./backend/syn/dct/outputs/` 目录下的综合网表文件 `$(TOP).v` 进行一致性验证，产生的报告保存 `./backend/formality/fm_dct.log` 文件中，在形式验证前，保证：

- 已成功完成DC综合
- `./backend/formality/script/` 文件夹下对应的tcl脚本已经根据要求修改

Formality验证环境清理

```
make clean_fm
```

该命令将删除 `./backend/formality/` 目录下产生的临时文件、报告日志等文件

自动化脚本命令使用说明

Python Env

```
pyauto cmd $1 $2 $3 $4
```

1. generate file list under the specified path

```
pyauto flstgen src dst
```

- src: rtl folder
- dst: destination file

2. copy file from filelist/folder to destination folder

```
pyauto cpfile src dst
```

- src: filelist or folder
- dst: destination folder

3. generate simfile filelist according to rtl and testbench

```
pyauto simfilegen src1 src2 dst
```

- src1: rtl folder
- src2: testbench file
- dst: destination file

4. generate testbench

```
pyauto tbgen src dst
```

- src: rtl file
- dst: destination folder

5. generate testbench for auto generating single port ram model

```
pyauto spramtbgen src dst
```

- src: rtl file
- dst: destination file

6. generate testbench for auto generating double port ram model

```
pyauto dpramtbgen src dst
```

- src: rtl file
- dst: destination file

7. generate ram model

```
pyauto ramgen param dst
```

- param: dwidth,awidth,dout_delay,port
- dst: destination file or folder

8. generate module top file

```
pyauto link src dst
```

- src: rtl file list
- dst: destination file

Makefile Env

```
make cmd SRC1 SRC2 DST
```

1. generate file list under the specified path

```
make flstgen SRC1= DST=
```

- SRC1: rtl folder
- DST: destination file

2. copy file from filelist/folder to destination folder

```
make cpfile SRC1= DST=
```

- SRC1: filelist or folder
- DST: destination folder

3. generate simfile filelist according to rtl and testbench

```
make simfilegen SRC1= SRC2= DST=
```

- SRC1: rtl folder
 - SRC2: testbench file
 - DST: destination file
4. generate testbench
- ```
make tbgen SRC1= DST=
```
- SRC1: rtl file
  - DST: destination folder
5. generate testbench for auto generating single port ram model
- ```
make spramtbgen SRC1= DST=
```
- SRC1: rtl file
 - DST: destination file
6. generate testbench for auto generating double port ram model
- ```
make dpramtbgen SRC1= DST=
```
- SRC1: rtl file
  - DST: destination file
7. generate ram model
- ```
make ramgen SRC1= DST=
```
- SRC1: dwidth,awidth,dout_delay,port
 - DST: destination file or folder
8. generate module top file
- ```
make link SRC1= DST=
```
- SRC1: rtl file list
  - DST: destination file

## 清空工程

---

```
make clearprj
```

该命令将完全清空下除了原始makefile模板文件以外的所有文件和目录，**非必要情况下不要使用此命令**